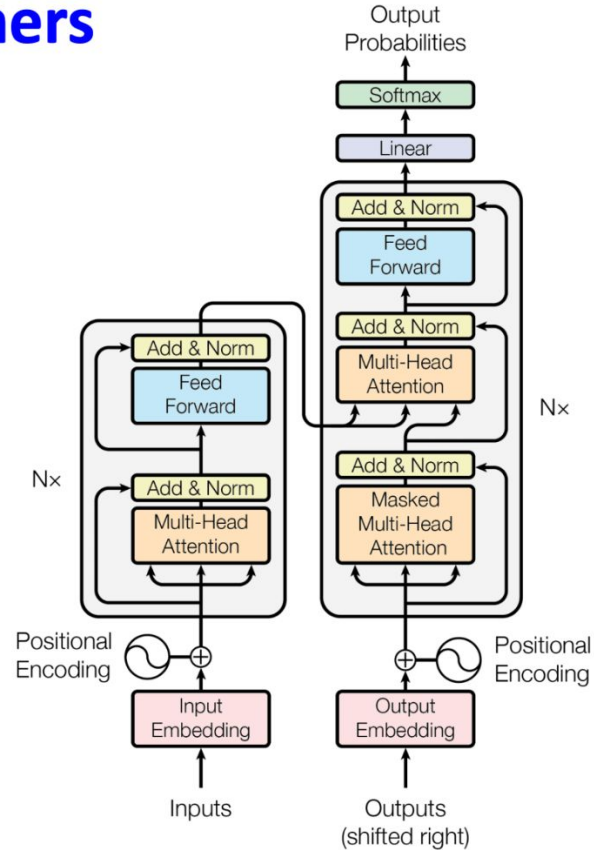


Join the lecture online on your dashboard.

Transformers: A New Paradigm

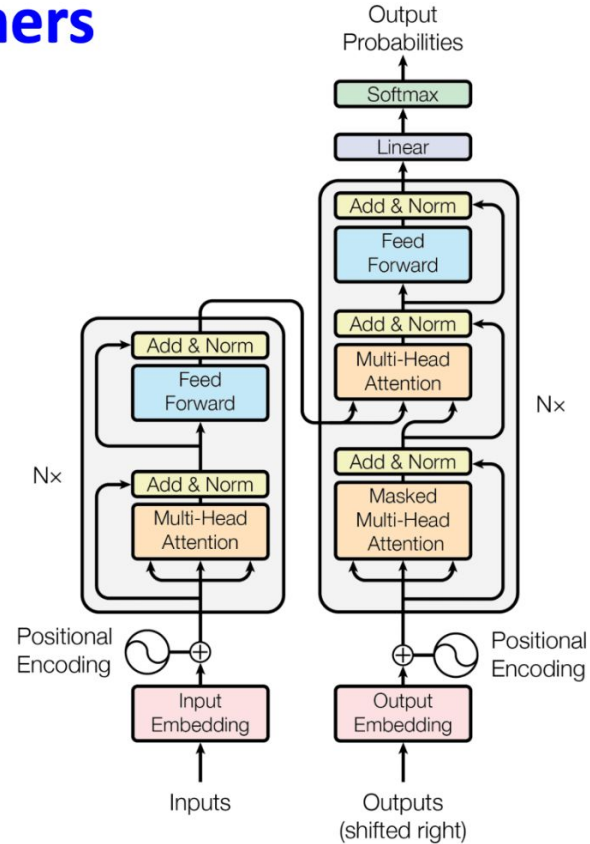
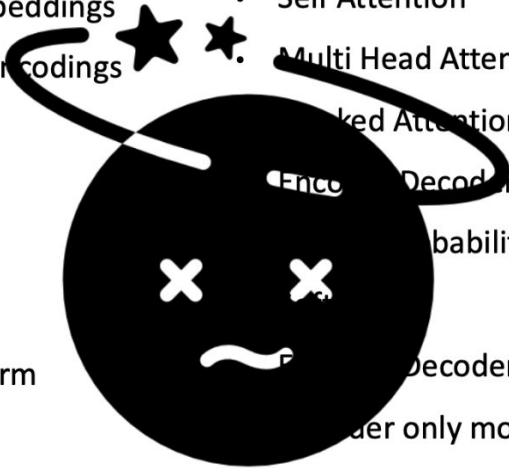
Transformers

- Tokenization
- Input Embeddings
- Position Encodings
- Residuals
- Query
- Key
- Value
- Add & Norm
- Encoder
- Decoder
- Attention
- Self Attention
- Multi Head Attention
- Masked Attention
- Encoder Decoder Attention
- Output Probabilities / Logits
- Softmax
- Encoder-Decoder models
- Decoder only models

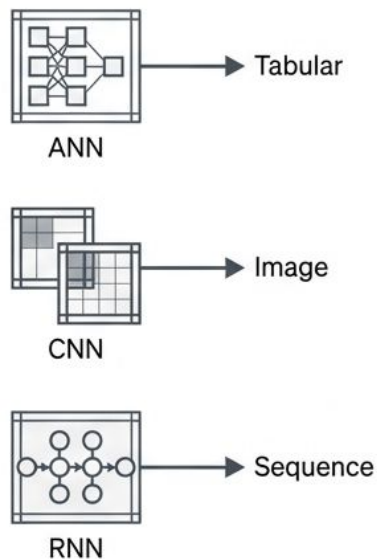


Transformers

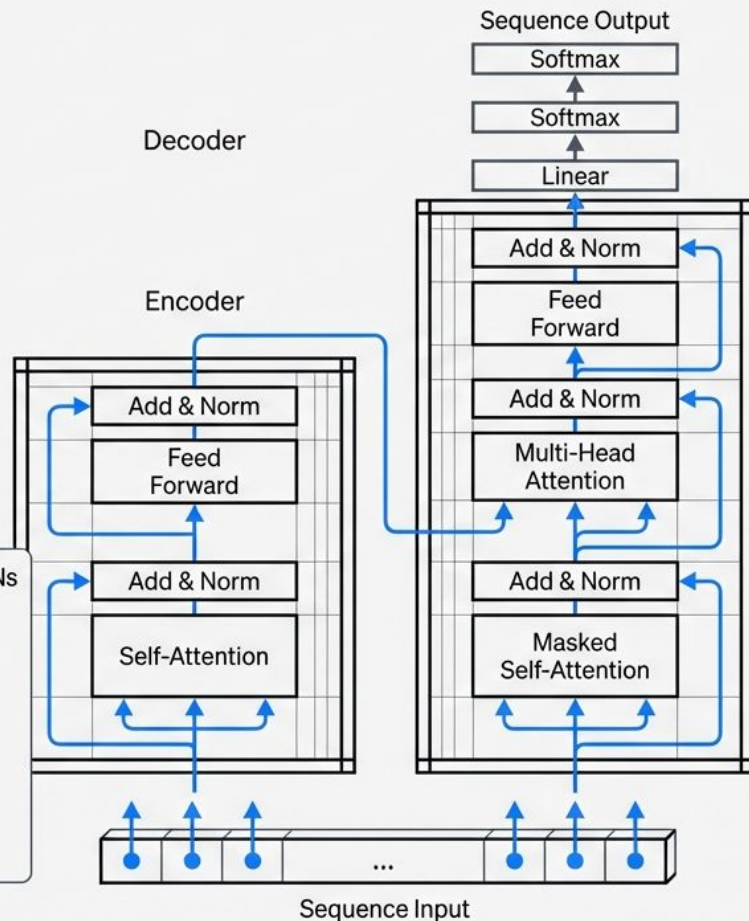
- Tokenization
- Input Embeddings
- Position Encodings
- Residuals
- Query
- Key
- Value
- Add & Norm
- Encoder
- Decoder
- Attention
- Self Attention
- Multi Head Attention
- Masked Attention
- Encoder-Decoder Attention
- Probabilities / Logits
- Encoder only models
- Decoder only models



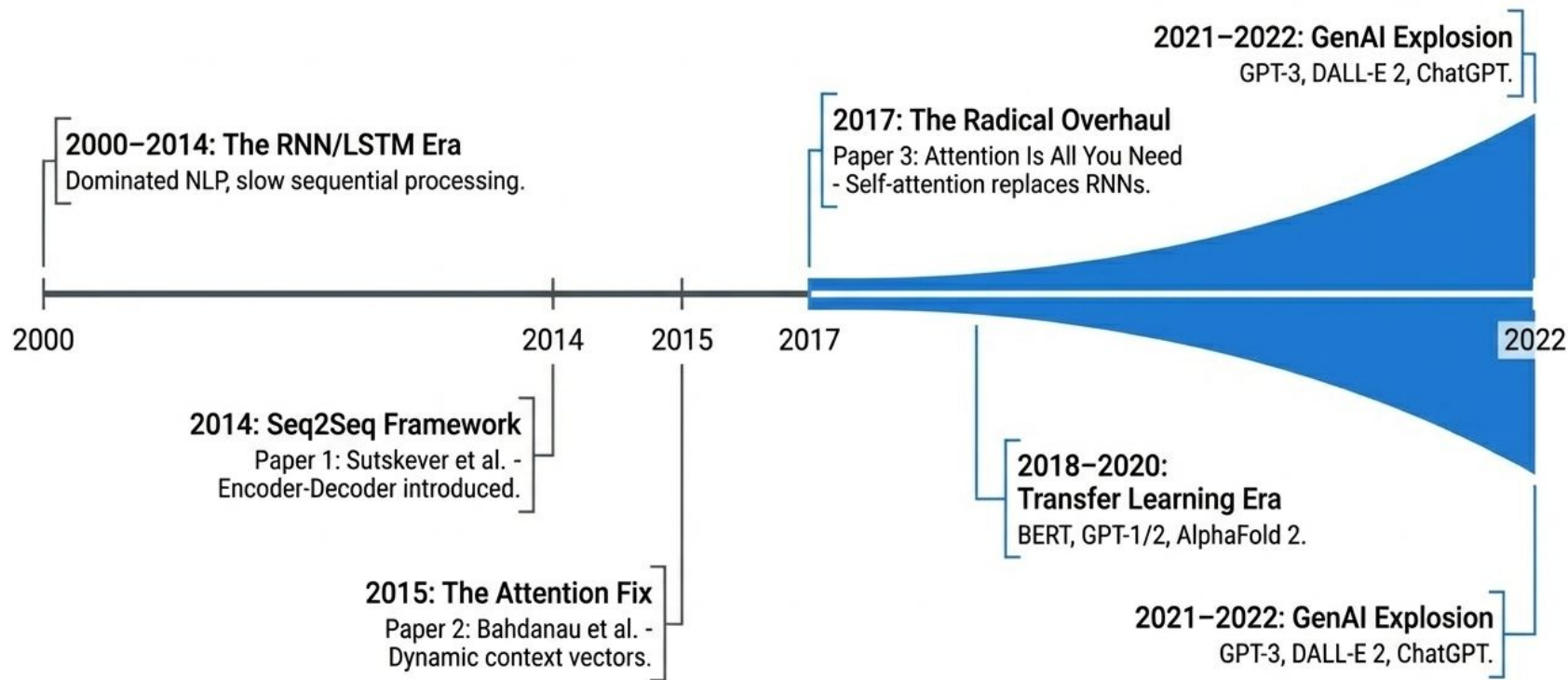
Beyond Specialized Architectures



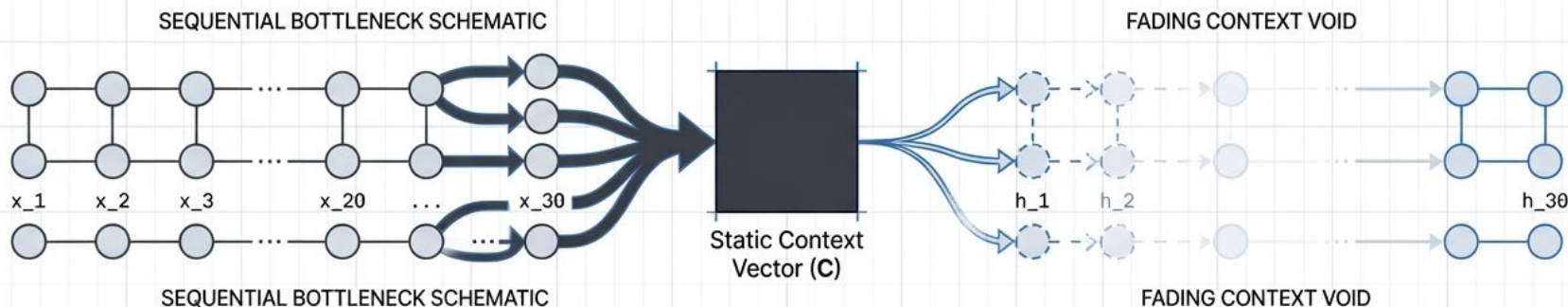
- Legacy State: ANNs for Tabular, CNNs for Images, RNNs/LSTMs for Text.
- The Transformer: A neural network architecture specifically engineered for Sequence-to-Sequence (Seq2Seq) tasks.
- The Difference: Retains the Encoder-Decoder structure but completely discards sequential processing.



The Evolution of Information Flow (2000–2022)

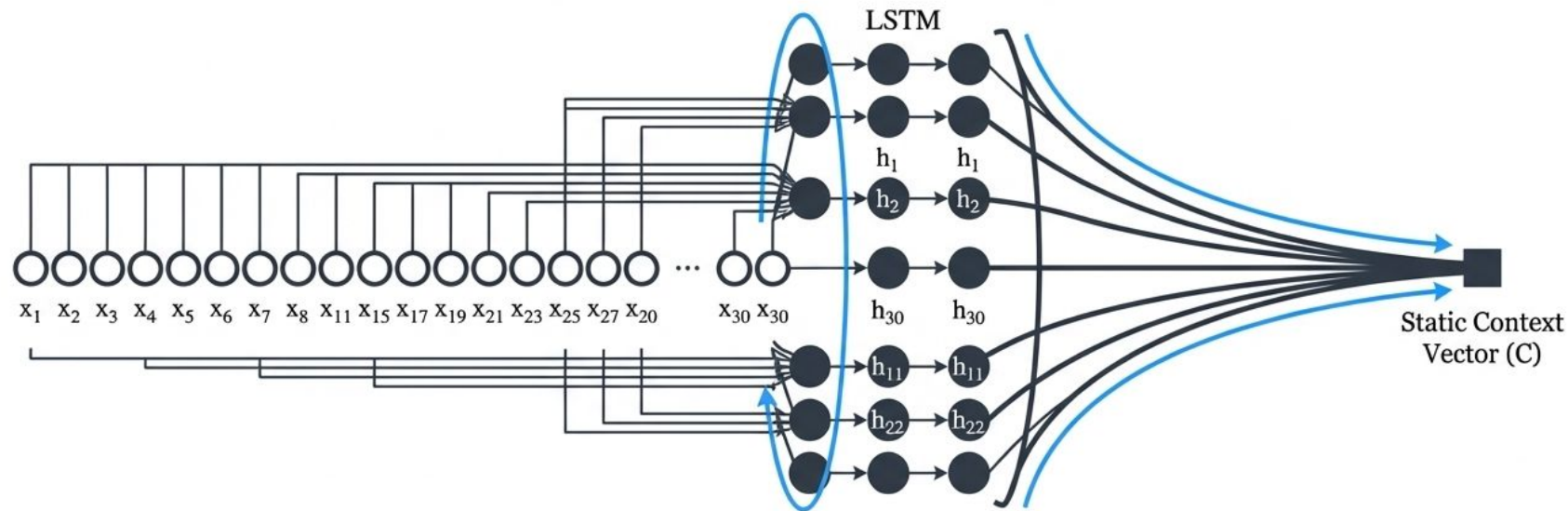


If human memory effortlessly stitches timelines together, why does the strict passage of time act as the primary enemy of legacy language models?



The Compression Limit: Because forcing an entire sequence into a **single, fixed-length context vector** mathematically guarantees the erasure of early context. In legacy architectures, time creates an unavoidable compression limit.

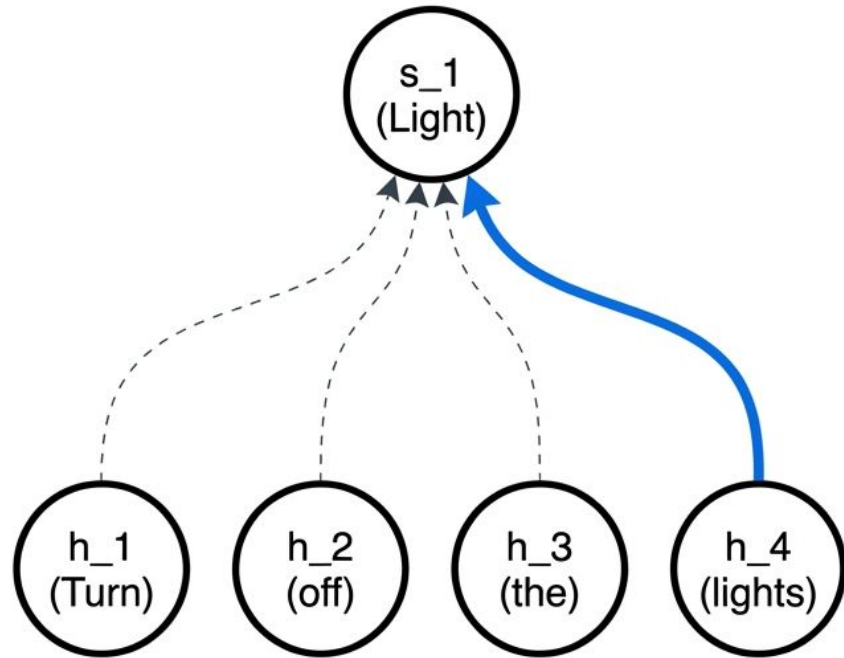
The Sequential Bottleneck



Core Concept

The Flaw	Data Loss	Sequential Limitation
Legacy Seq2Seq models force an entire sequence (e.g., 30+ words) into a single, fixed-length context vector.	As input grows, the model fails to retain early context. The mathematical representation physically cannot hold the full semantic essence.	Words must be processed step-by-step ($t_1 \rightarrow t_2 \rightarrow t_3$), making parallel computing impossible.

The Incremental Fix: Dynamic Attention



The Mechanism: Instead of one static vector, the decoder requests a dynamic context vector at every time step.

Attention Weights: The model mathematically calculates which input words are most relevant to the current output word.

Result: Solved the context-loss problem for long sentences, but the underlying architecture remained locked to slow, sequential RNNs.

In 2017, a paper was published called 'Attention Is All You Need'. They proposed a radical idea: Why do we even need the RNN or the LSTM?.

They built a network based solely on attention mechanisms, dispensing with recurrence entirely

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

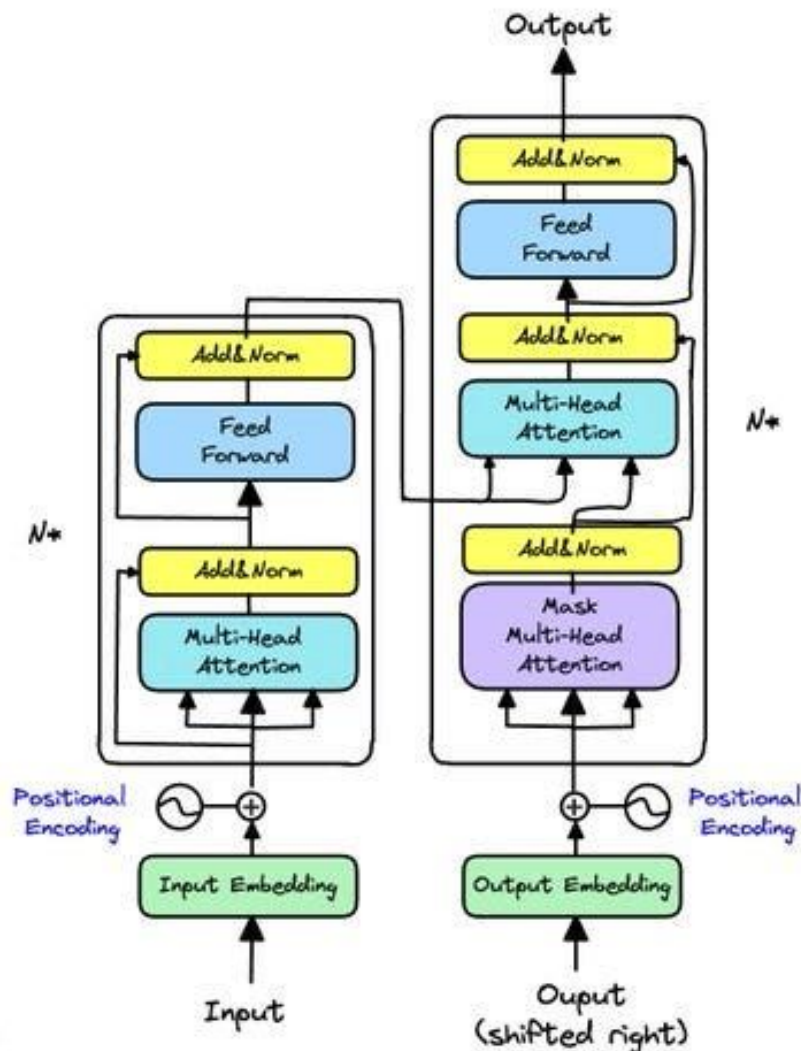
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

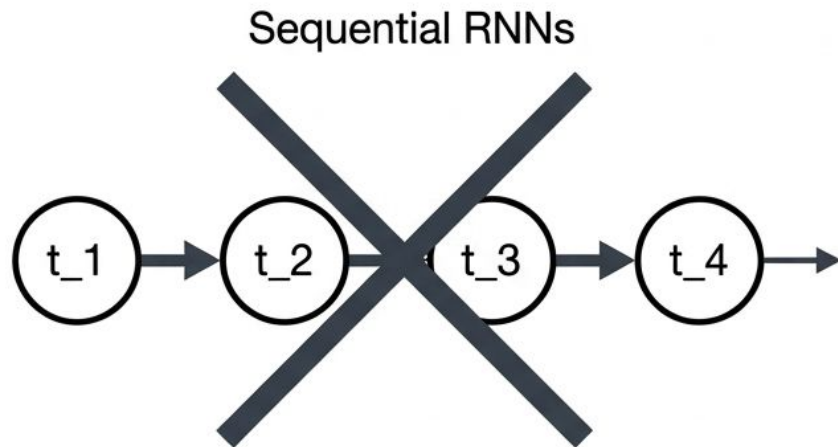
*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.



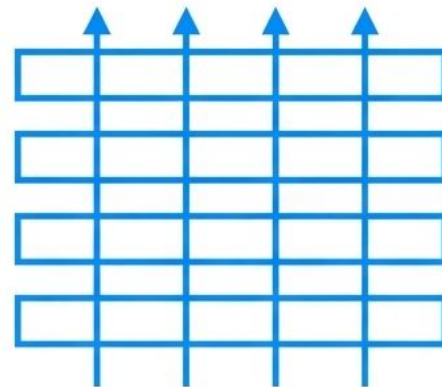
The Radical Shift: Attention Is All You Need



The 2017 Breakthrough:

Researchers completely discarded LSTMs and RNNs.

Self-Attention Mechanism



Self-Attention:

A novel mechanism allowing the encoder to process all words in a sentence simultaneously.

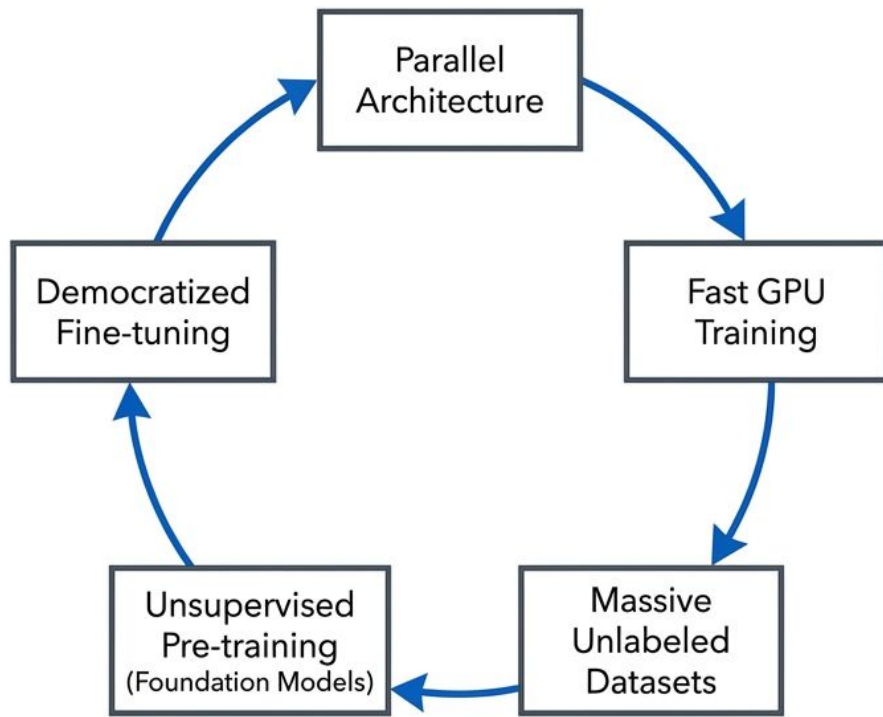
The Leap:

This was not an incremental tweak. It was a structural overhaul that decoupled sequence length from processing time.

Architectural Benchmarking

Dimension	Legacy (RNN/LSTM)	Modern (Transformer)
Processing Method	Word-by-word (Time-dependent)	Entire Sequence (Simultaneous)
Training Speed	Slow (Hardware bottlenecked)	Highly Parallel (GPU optimized)
Scalability	Limited by sequence length	Massive (Billions of parameters)
Transfer Learning	Practically impossible	Native (Foundation Models)

The Transfer Learning Flywheel

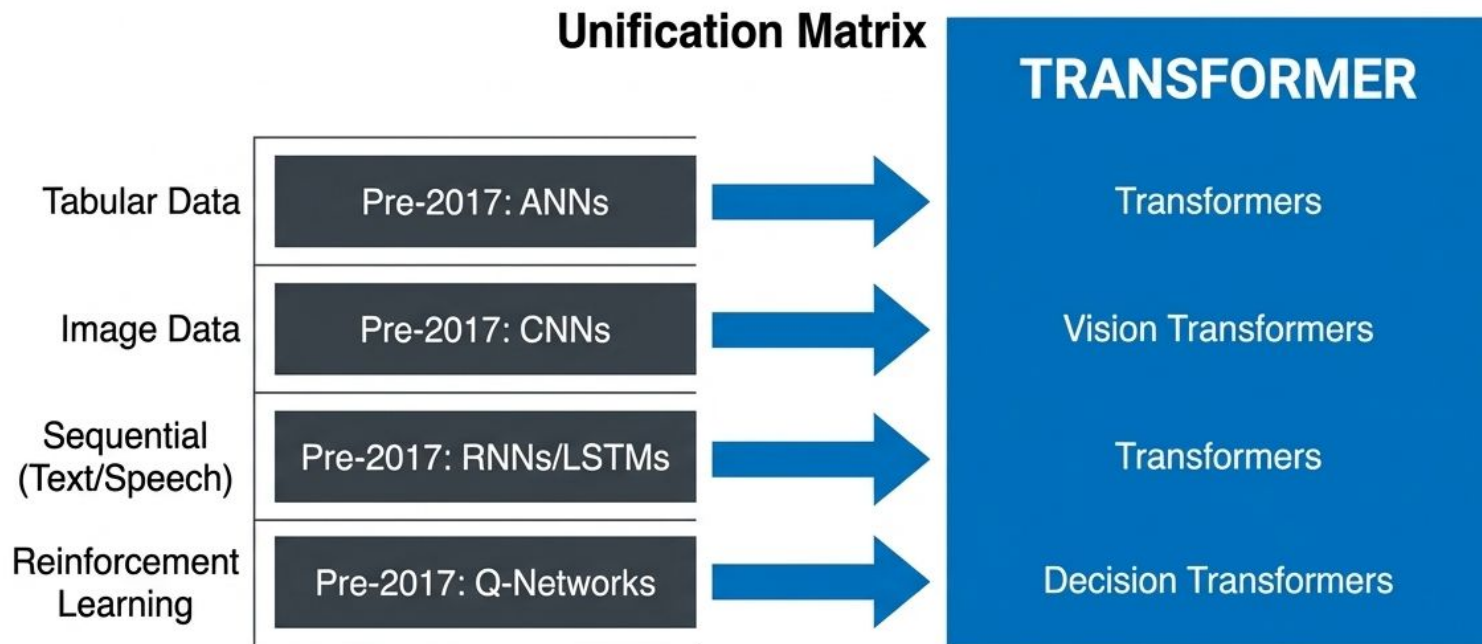


Democratizing AI

Because Transformers train in parallel, they can ingest massive web-scale datasets to form Foundation Models (like BERT or GPT).

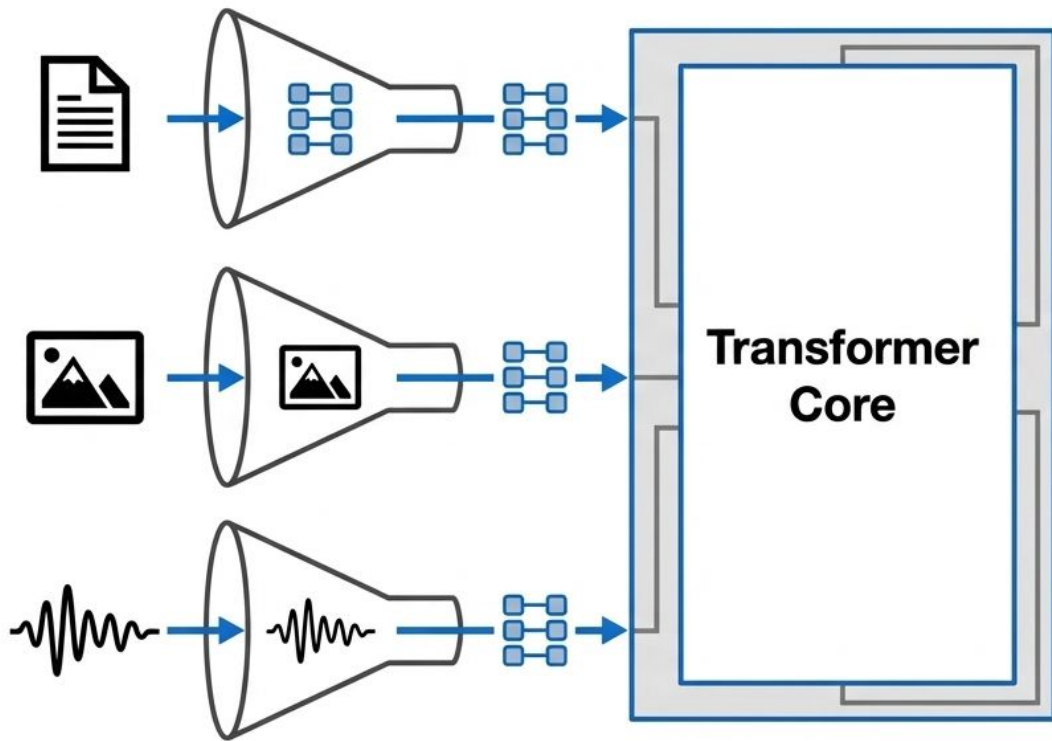
Small teams can now download these pre-trained models and fine-tune them on small datasets with minimal compute (e.g., via Hugging Face), a feat impossible with RNNs.

The Unification of Deep Learning



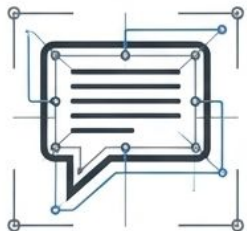
Takeaway: We are witnessing a paradigm shift from domain-specific architectures to a singular, universal computation engine.

Multimodal By Design



- **Universal Representation**
Whether handling pixels, sound waves, or text, Transformers operate on abstracted representations (embeddings).
- **Cross-Modal Generation**
The architecture allows for text-in to image-out (DALL-E), image-in to text-out (Visual Search), or text-in to video-out (RunwayML).
- **Flexibility**
Can be configured as Encoder-only (BERT) or Decoder-only (GPT) based on the exact generation required.

Landmark Implementations



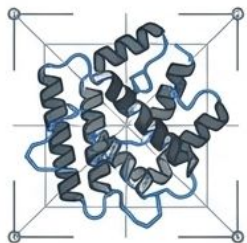
ChatGPT (Text/GenAI)

Built on Decoder-only GPT architecture. Revolutionized conversational AI and rapid text generation.



DALL-E 2 (Cross-Modal)

Integrates Transformers with diffusion mechanisms to map natural language prompts to high-fidelity image generation.



AlphaFold 2 (Structural Biology)

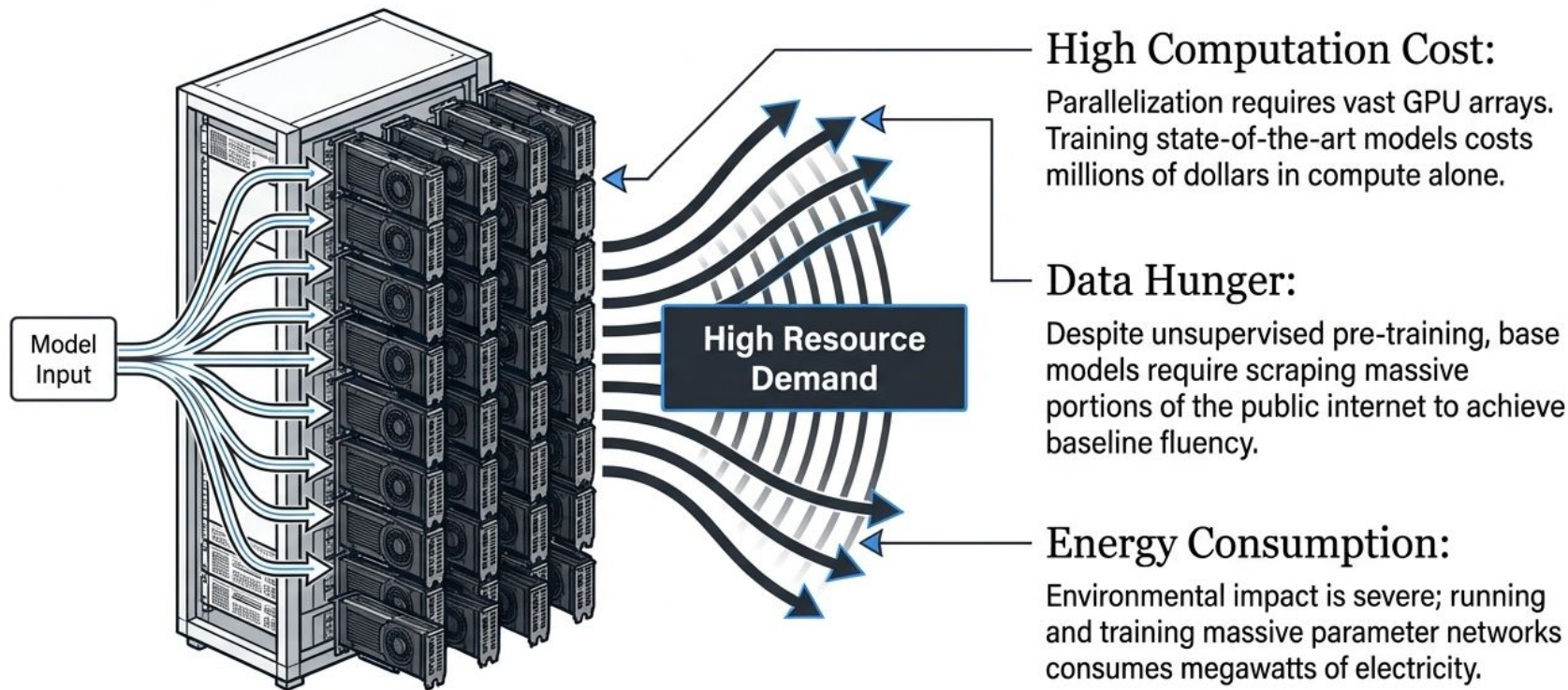
DeepMind's breakthrough mapping 3D protein structures, fundamentally accelerating scientific research.



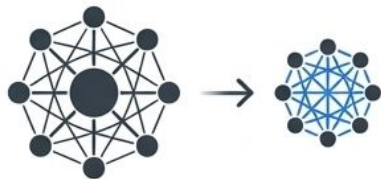
OpenAI Codex (Code Gen)

Translates natural language into functional programming code (the engine behind GitHub Copilot).

The Hardware & Compute Trade-offs



Future Research Vectors



Efficiency & Compression

Techniques like Pruning, Quantization, and Knowledge Distillation to run LLMs on edge devices.



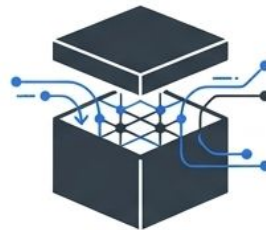
Domain-Specific Models

Moving from generalized Chatbots to highly specialized models trained exclusively on proprietary data (e.g., LegalGPT, DoctorGPT).



Multilingual Localization

Breaking the English-centric barrier by training base models from scratch in regional languages (e.g., Krutrim AI in Hindi).

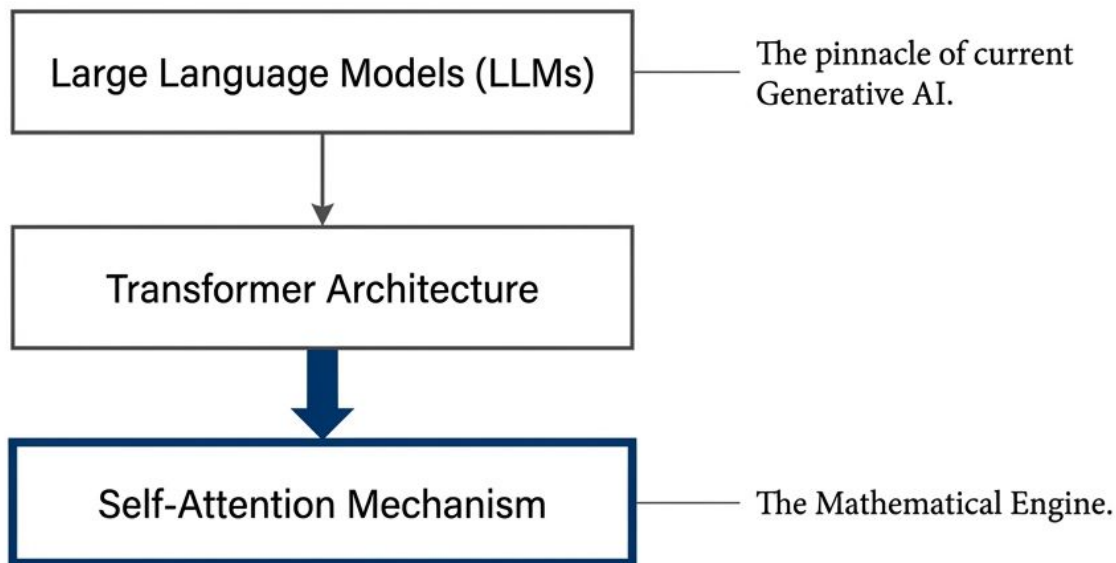


White-Box Explainability

Developing tools to dissect and interpret attention weights to trace exactly why a model made a specific decision.

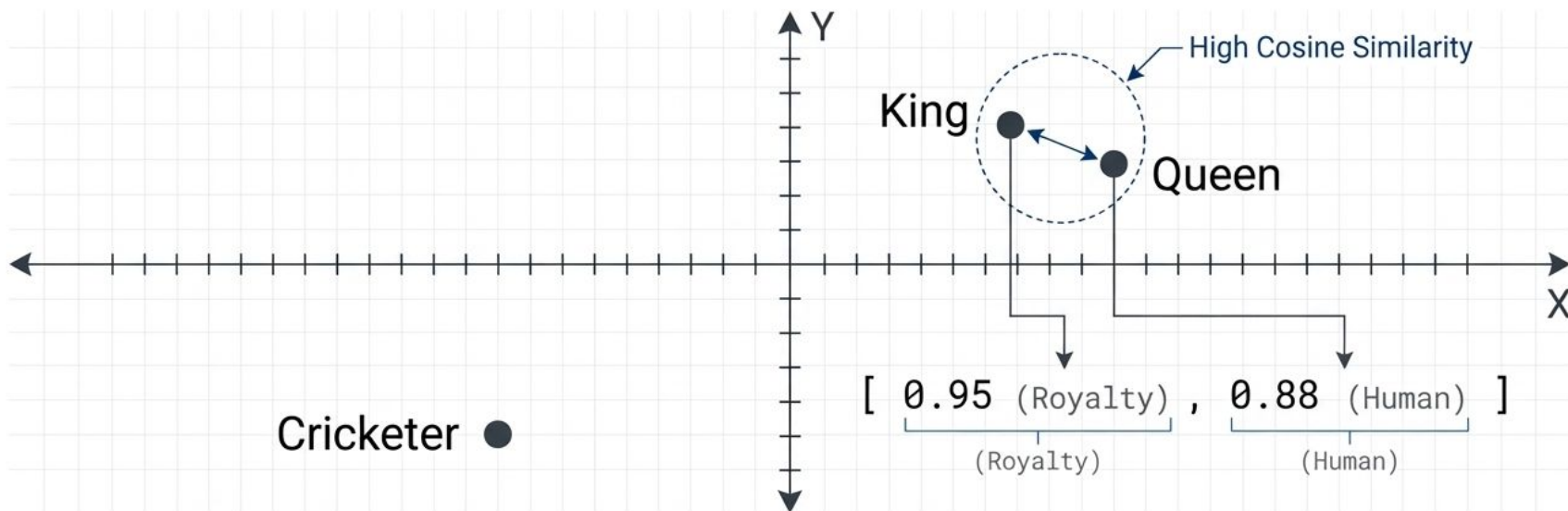
Deep Dive into its Architecture: Self Attention

The Foundation of Generative AI Architecture



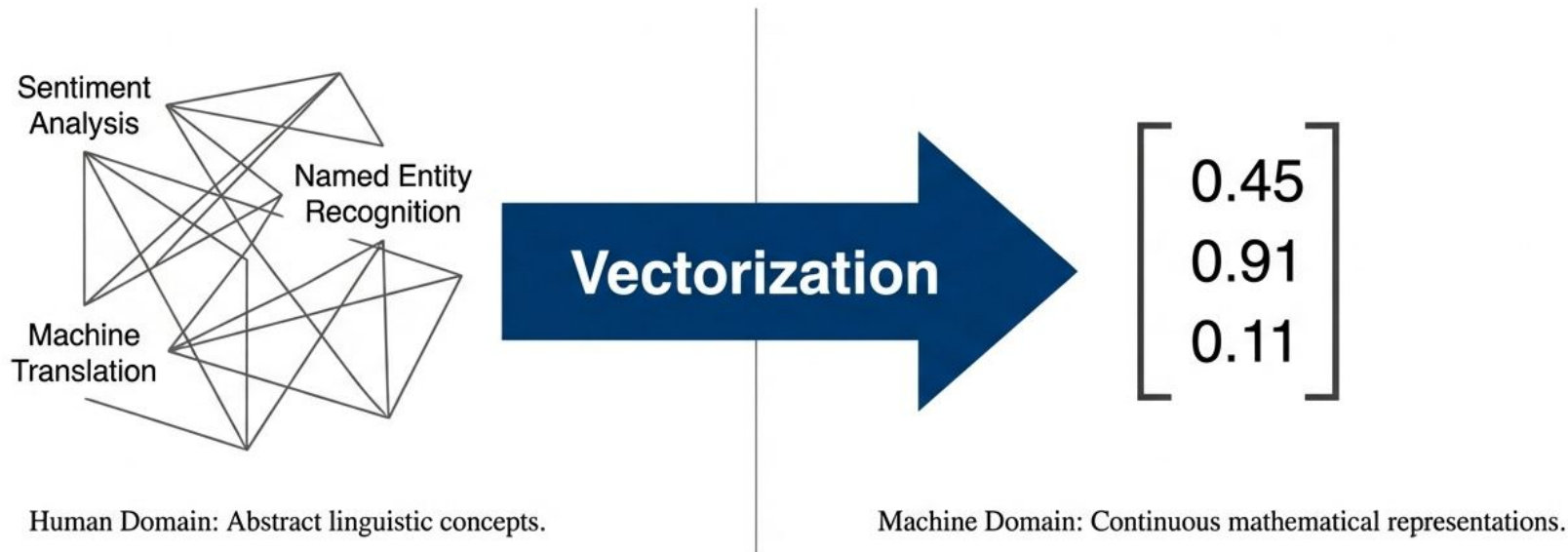
Mastering the self-attention mechanism provides the mathematical and intuitive foundation for all modern Generative AI systems. To understand LLMs, you must first master this core algorithmic layer.

How does a machine, fundamentally blind to abstract human concepts, mathematically deduce that a “King” and a “Queen” are intimately related?



Spatial Semantics: By translating meaning into n-dimensional coordinates. In this latent space, semantic similarity is no longer an abstract human idea—it is a physical, measurable, and computable proximity.

The Core NLP Bottleneck: The Semantic Gap



The primary prerequisite for any NLP task is the highly efficient translation of text into numerical vectors. Computers are blind to abstract symbols. They process continuous mathematical representations. Meaning must be converted into math.

Era I: One-Hot Encoding (Sparse Representation)

Vocabulary = [Mat, Cat, Rat]

$$\text{Mat} = \begin{bmatrix} 1, 0, 0 \end{bmatrix}$$

$$\text{Cat} = \begin{bmatrix} 0, 1, 0 \end{bmatrix}$$

$$\text{Rat} = \begin{bmatrix} 0, 0, 1 \end{bmatrix}$$

Diagnostic Callout: Creates **massively** sparse, orthogonal vectors.

Sentence Example:

Mat Cat


$$\begin{bmatrix} 1, 0, 0 \\ 0, 1, 0 \end{bmatrix}$$

Vocabulary scaling results in extreme computational bloat with zero semantic correlation between similar words.

Era II: Bag of Words (Frequency Representation)

Sentence 1:

Mat Cat Mat



$[2, 1, 0]$

Sentence 2:

Cat Rat Rat

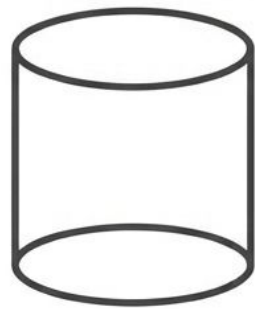


$[0, 1, 2]$

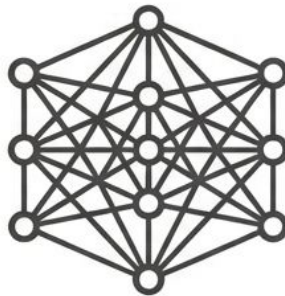
Moves from binary existence to frequency tracking.

Diagnostic Callout: Captures occurrence and frequency, yet remains completely blind to word order, syntax, and underlying semantic context. The structural meaning of the sentence is destroyed.

The Semantic Leap: Dense Word Embeddings



Massive Corpus
(e.g., Wikipedia)



Neural Network
Training

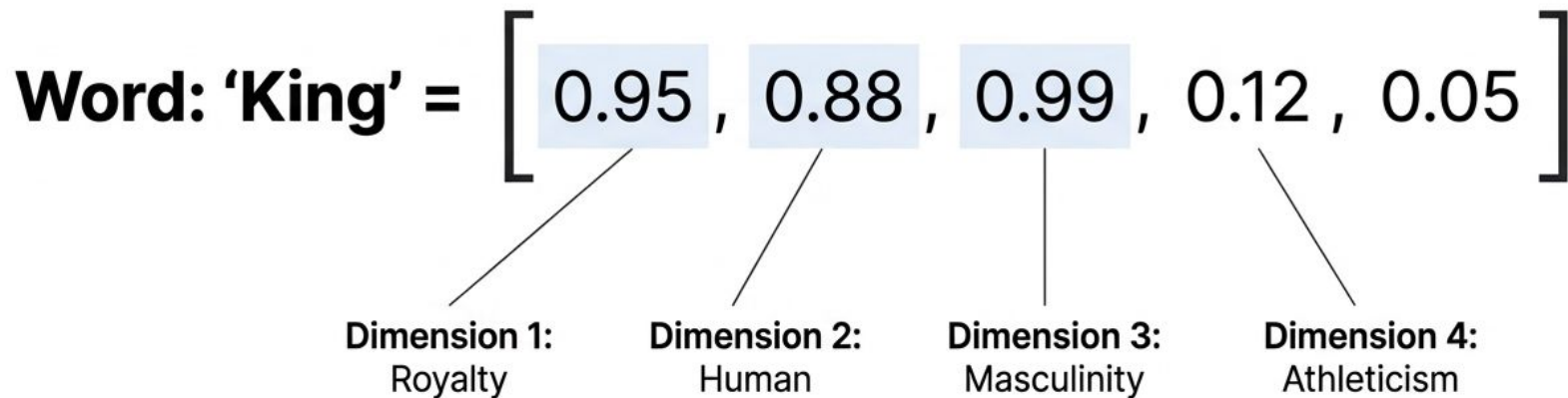


N-Dimensional Dense Vectors
($n = 64, 256, 512$)

Word Embeddings represent a paradigm shift: forcing neural networks to encode the semantic meaning and usage patterns of a word into a dense numerical array.

Mechanism: Rather than counting frequencies, the network learns representations by observing the contexts in which words naturally appear across millions of documents.

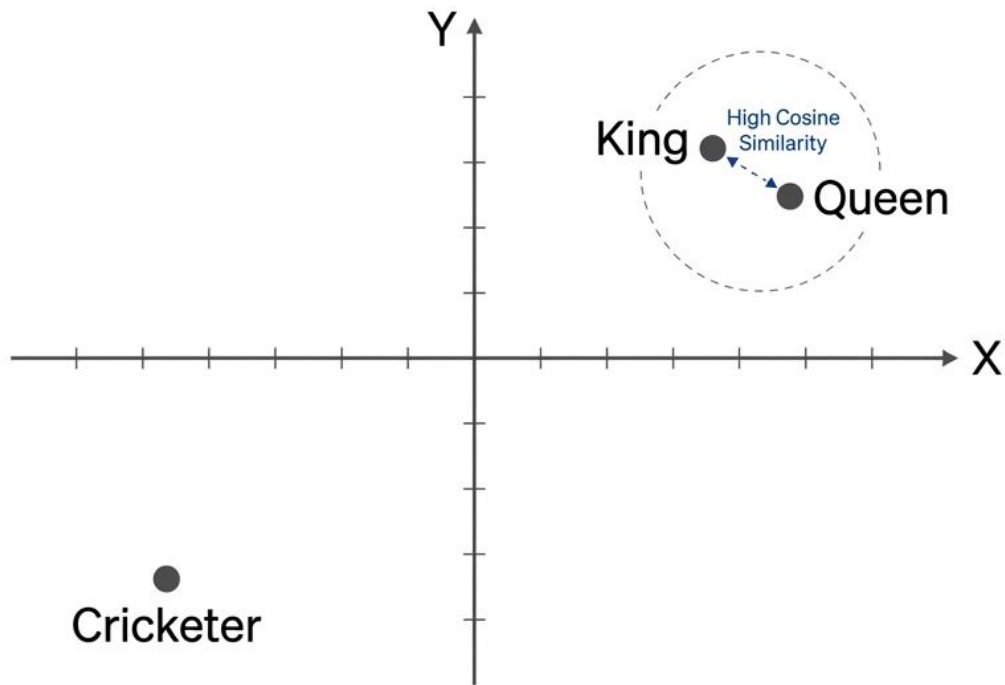
The Anatomy of a Semantic Vector



Though dimensions are latent and machine-derived, they map to real-world semantic traits.

Comparative Note: A word like "Queen" will trigger nearly identical activations for Royalty and Human, but diverge sharply on Masculinity. A word like "Cricketer" will spike on Athleticism.

Geometric Intuition in Latent Space



In this mathematical space, meaning becomes spatial.

Rule: Similar words exhibit high cosine similarity and cluster together naturally. Dissimilar words mathematically distance themselves.

The Fatal Flaw of Static Embeddings

**Static Word Vector =
Average(Context A
+ Context B + Context C)**

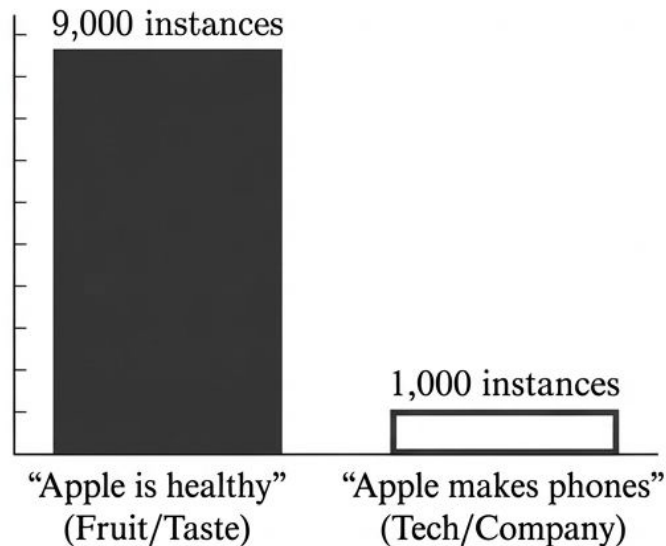


Word embeddings possess a critical limitation: they are generated once and never change. They are static.

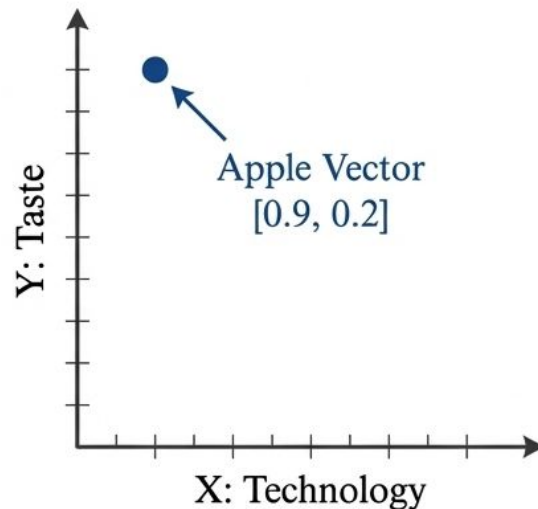
A static embedding maps a single word to a single vector, forcing it to represent its statistical average across all training data, regardless of its immediate usage.

The 'Apple' Paradox: Training Data Skew

The Input Data Bias



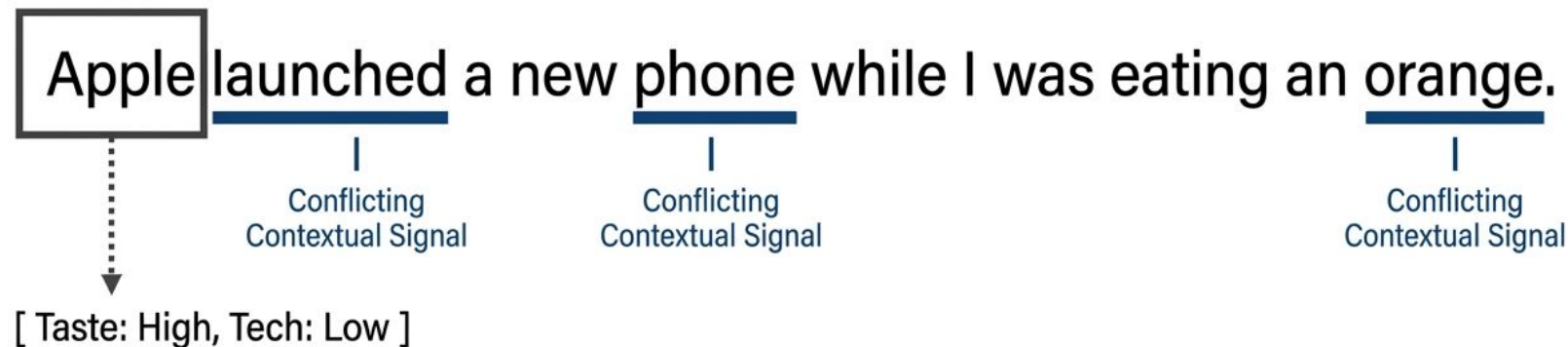
The Resulting Vector



Because the training corpus heavily favors "Apple" as a fruit, the static vector permanently skews its highest values toward the "Taste" dimension.

Consequence: The technology dimension is mathematically suppressed.

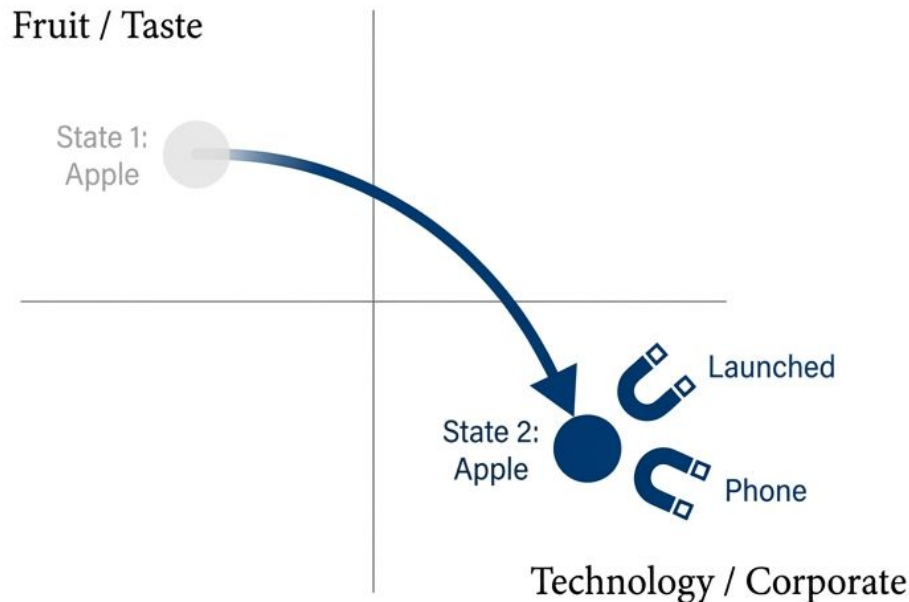
The Contextual Collision (System Failure)



In downstream tasks like translation, the static vector fails.

Analysis: The system blindly perceives 'Apple' as a fruit due to its static training average, completely ignoring the overwhelming local contextual clues of 'launched' and 'phone'.

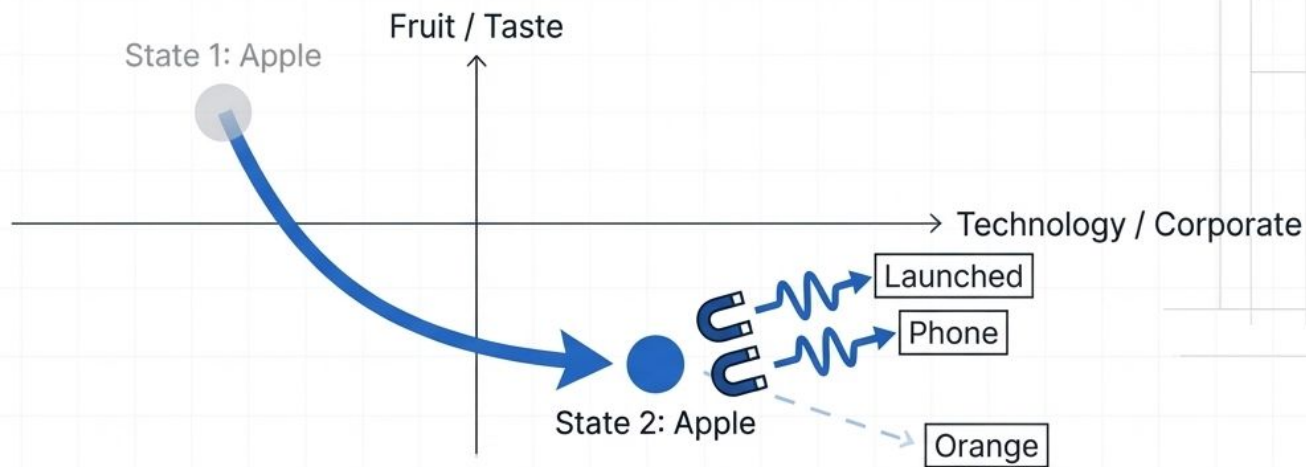
The Requirement for Dynamic Semantics



The Goal: Contextual Embeddings.

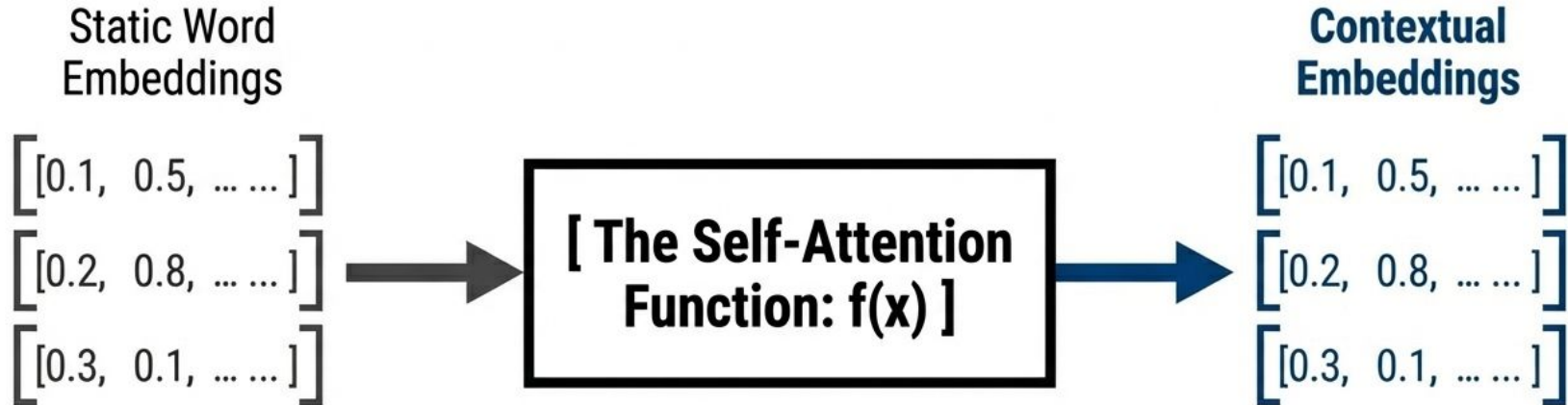
We require a dynamic system capable of adjusting a word's vector coordinates on the fly by 'looking' at the surrounding neighborhood of words in that specific sentence.

If static embeddings are rigid mathematical coordinates, what mechanism acts as the 'gravity' to pull a word's meaning toward its neighbors?



The **Self-Attention Mechanism**: It acts as an algorithmic upgrade layer. It calculates intra-sentence relevance scores, acting as “semantic gravity” to actively re-weight and mutate a word’s vector coordinates *on the fly* based entirely on its immediate neighbors.

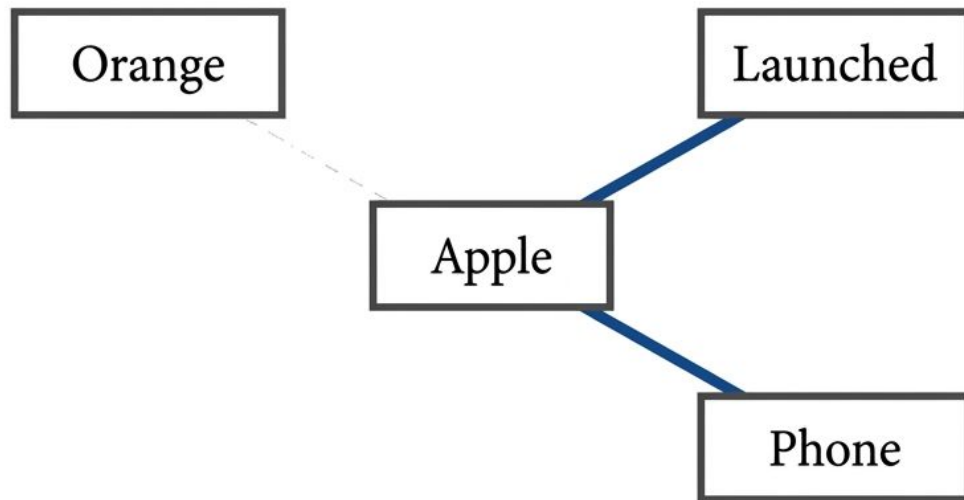
Enter the Self-Attention Mechanism



Self-Attention is a transformative mathematical function. It is not an alternative to embeddings; it is an upgrade layer.

Mechanism: It ingests rigid, static vectors and mathematically re-weights them based on intra-sentence relationships to generate smart, contextual outputs.

Mathematical Re-weighting in Action



The mechanism calculates relevance scores between every word pair in the sequence.

'Apple' recognizes high semantic relevance with 'Phone', forcing its vector to spike the 'Technology' dimension and suppress the 'Taste' dimension, ignoring the 'Orange' trap.

Enrichment of Static Embeddings

The Limitation of Static Embeddings

Sentence A: Money Bank Grows

"Bank" = Financial Institution



$[0.4, -0.1, 0.8]$

Sentence B: River Bank Flows

"Bank" = Edge of a River



$[0.4, -0.1, 0.8]$

The Algorithmic Bottleneck:

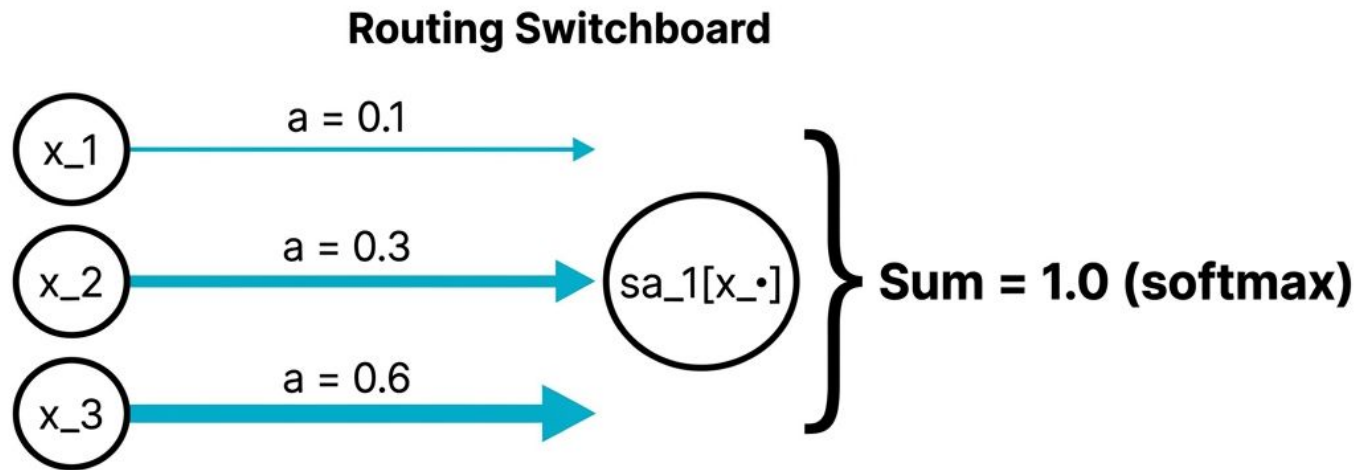
Traditional word embeddings (Word2Vec, GloVe) are static lookups.
 E_{bank} yields the exact same N-dimensional vector regardless of surrounding tokens.

The Requirement: Contextual Embeddings

We need a dynamic representation where the numerical vector shifts based on surrounding context.

$$E_{\text{bank}}(\text{Context: Money}) \neq E_{\text{bank}}(\text{Context: River})$$

Self-Attention as Dynamic Routing

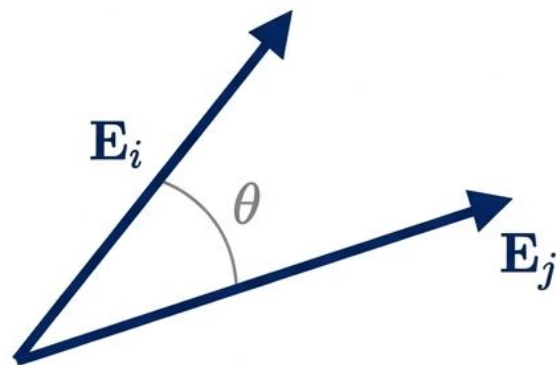


A standard network applies fixed weights. **Self-attention dynamically routes input values** in different proportions to create an output. The **weights are positive**, sum to one and change based on the inputs themselves.

$$\mathbf{E}_{\text{bank_new}} = (0.7 \times \mathbf{E}_{\text{money}}) + (0.2 \times \mathbf{E}_{\text{bank}}) + (0.1 \times \mathbf{E}_{\text{grows}})$$

Differentiability	Dynamism
The representation of “Bank” is now fundamentally altered by the presence of “Money” and “Grows”. It absorbs its context.	If the sequence changes to “River Bank Flows”, the constituent weights shift entirely (e.g., heavy weight on “River”), generating a mathematically distinct N-dimensional vector.

Quantifying Context: Dot Products as Similarity Scores



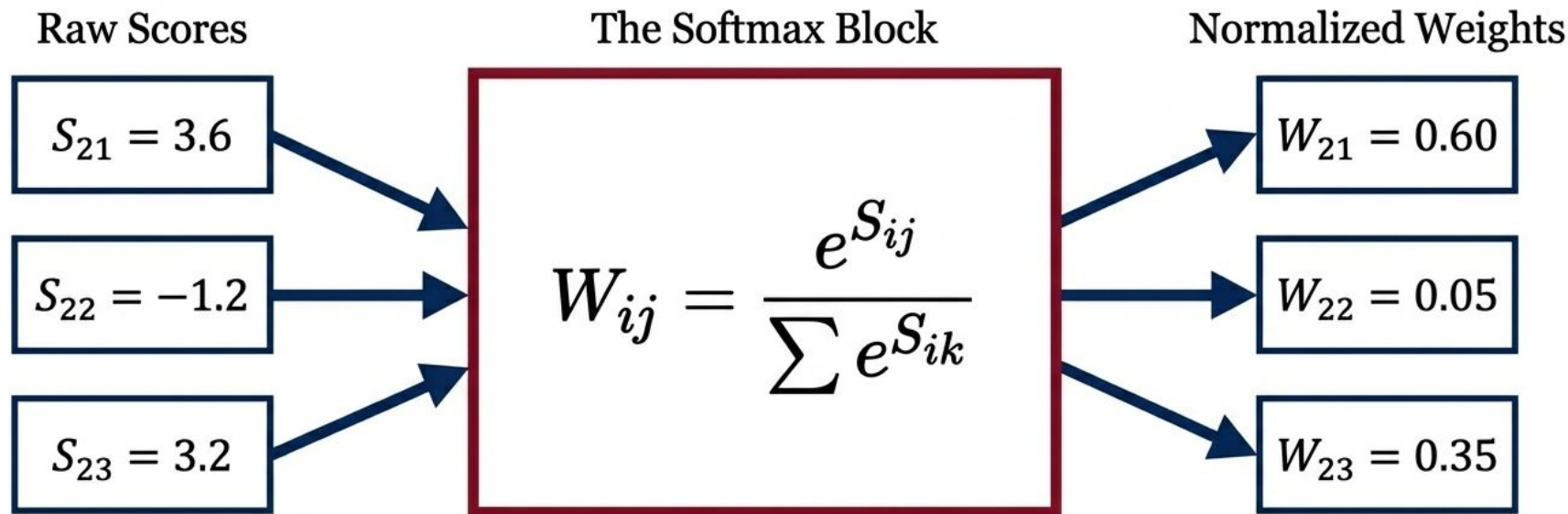
$$S_{ij} = \mathbf{E}_i \cdot \mathbf{E}_j$$

S_{ij} represents the raw similarity score between the target word i and context word j .

$S_{21} = \mathbf{E}_{\text{bank}} \cdot \mathbf{E}_{\text{money}}$	(Similarity with word 1)
$S_{22} = \mathbf{E}_{\text{bank}} \cdot \mathbf{E}_{\text{bank}}$	(Self-similarity)
$S_{23} = \mathbf{E}_{\text{bank}} \cdot \mathbf{E}_{\text{grows}}$	(Similarity with word 3)

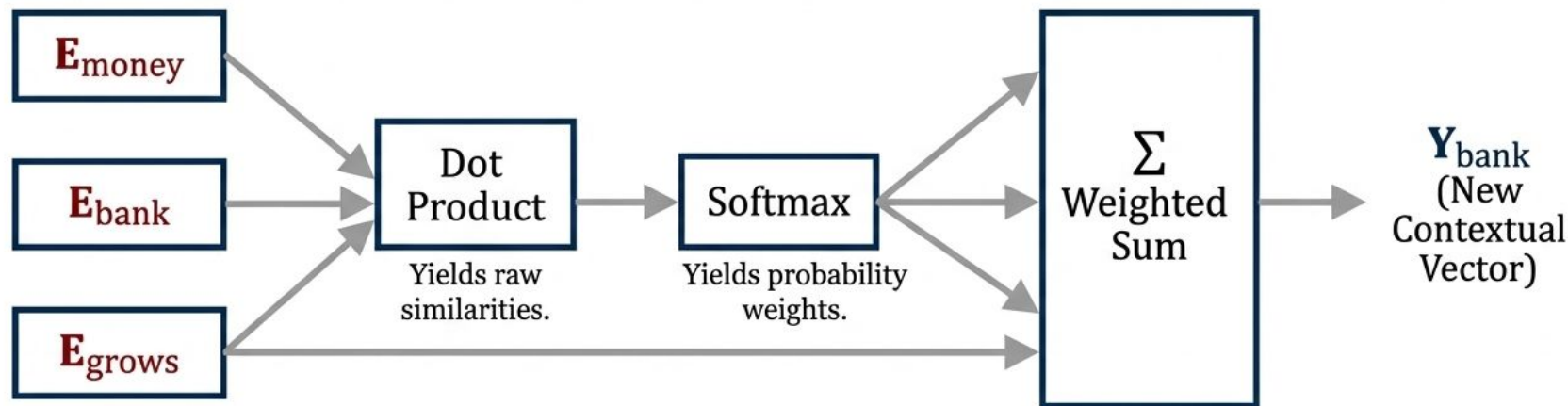
Algorithmic Note: Dot products yield unbounded scalars (ranging from $-\infty$ to ∞). They cannot be used as weights directly.

Normalizing Similarity: The Softmax Transformation



Mathematical Property: $\sum W_{2j} = 1.0$. Mapping unbounded scores into a strict probability distribution ensures the newly generated contextual vector maintains a stable magnitude, preventing exploding values deep within the neural network.

Constructing the Contextual Vector: The Final Aggregation



$$\mathbf{Y}_{\text{bank}} = (W_{21} \times \mathbf{E}_{\text{money}}) + (W_{22} \times \mathbf{E}_{\text{bank}}) + (W_{23} \times \mathbf{E}_{\text{grows}})$$

This three-step anatomy (Compare -> Normalize -> Synthesize) generates $\mathbf{Y}_i$, a dynamic contextual embedding for every word in the sequence.

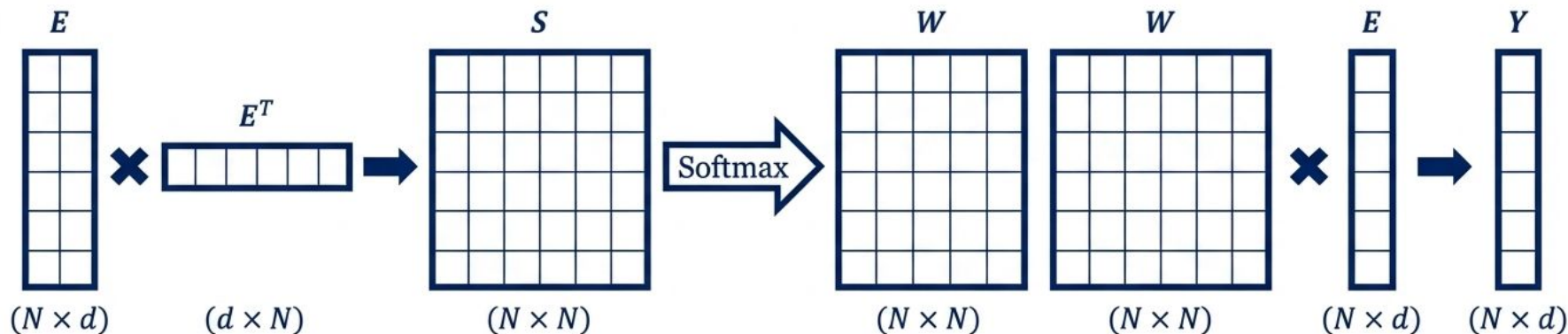
The Hardware Advantage: Matrix Parallelization

Let E be the $(N \times d)$ matrix of all sequence embeddings.

Step 1: Compute All Similarities
 $S = E \times E^T$

Step 2: Normalize (Row-wise)
 $W = \text{Softmax}(S)$

Step 3: Compute All Contextual Vectors
 $Y = W \times E$



Zero sequential dependencies. 3 words or 3,000 words process in a single parallelized matrix operation

The Critical Flaw: Zero Learnable Parameters

The Operations Analyzed

- ☒ **Dot Product** -> Pure vector math; no weights/biases.
- ☒ **Softmax** -> Mathematical constant; no weights/biases.
- ☒ **Weighted Sum** -> Pure aggregation; no weights/biases.

The “Piece of Cake” Problem

Premise:

Because the system lacks parameters (W, b), it generates *General* contextual embeddings. It cannot adapt to specific NLP objectives.

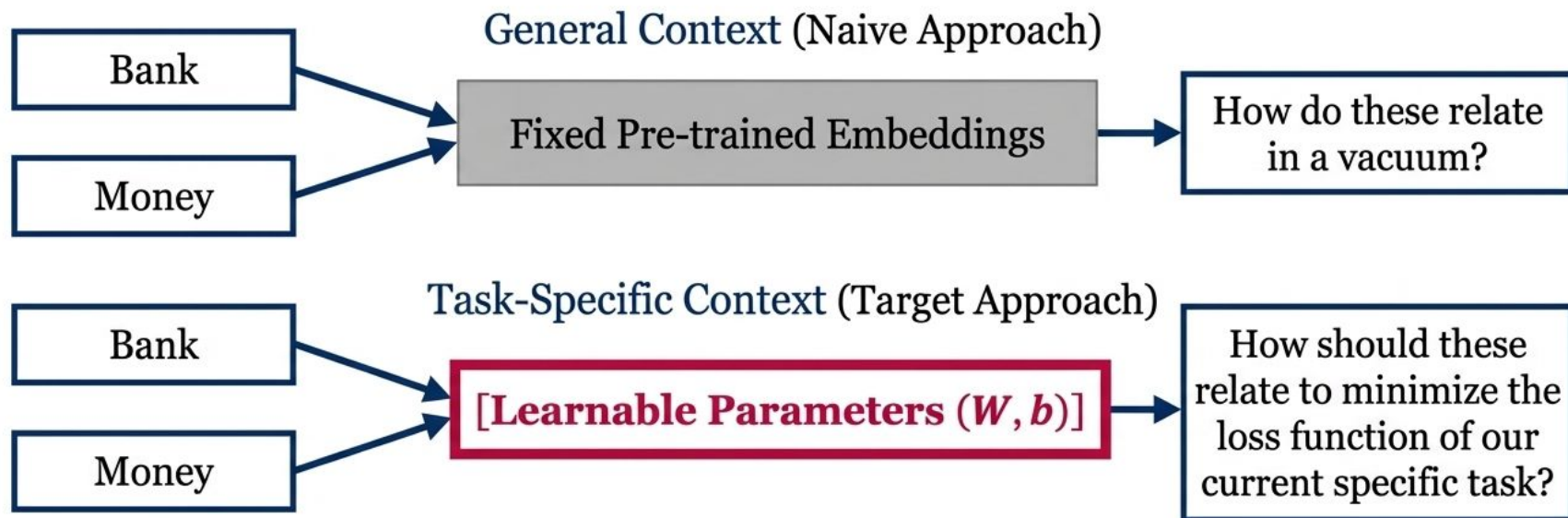
Literal Output:

A static dot-product mapping ‘Piece of cake’ for Translation will invariably output literal tokens (‘**a slice of baked dessert**’).

The Failure:

It entirely lacks the capacity to adapt to the objective of identifying task-specific idioms (‘**an easy task**’).

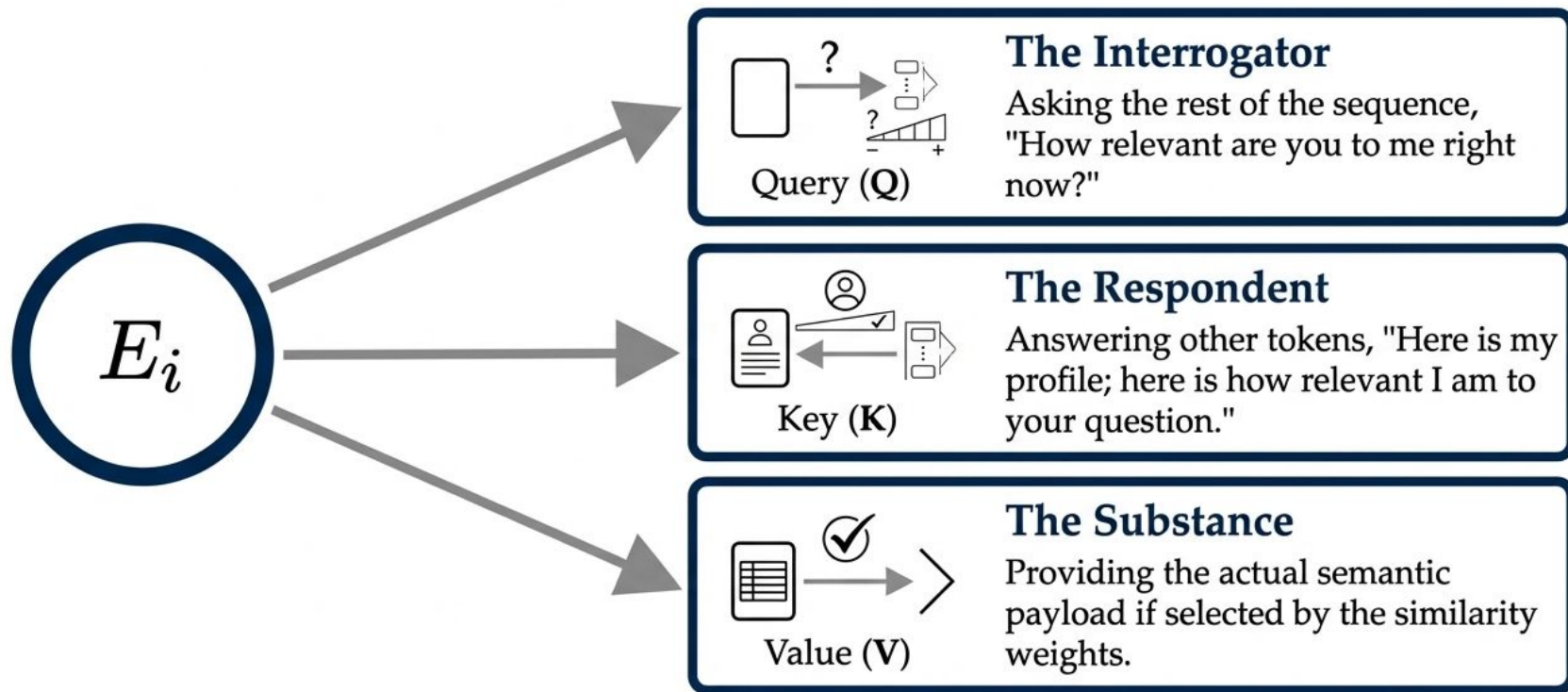
The Requirement: Task-Specific Contextualization



The Objective

We must introduce Learnable Weight Matrices into the self-attention flow without destroying its parallelizable matrix architecture. The model must learn *how* to pay attention based on the training data.

Deconstructing Attention: The Three Roles of a Word Vector



Forcing a single raw representation (E_i) to execute all three distinct roles perfectly is a mathematical impossibility. We must separate these concerns.

The Architecture of Retrieval: Query, Key, Value (QKV)

Concept	The Database Analogy	The Self-Attention Role
Query (Q)	The Search Bar entry. What you are looking for based on your current state.	The vector used to calculate similarity against others.
Key (K)	The Document Index/Title. How the document advertises itself to the search engine.	The vector compared against incoming Queries.
Value (V)	The Document Body. The actual content returned if the Query matches the Key.	The vector used to compute the final weighted sum.

In Task-Specific Self-Attention, every single word generates its own Query, its own Key, and its own Value.

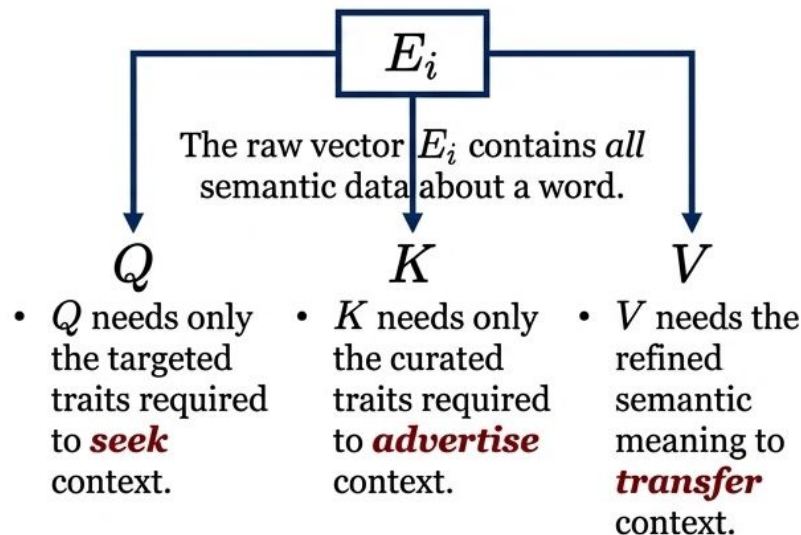
The Fallacy of Shared Vectors

The “Autobiography” Analogy



- Imagine searching a database where every user simply uploads a 300-page unstructured autobiography.
- You search using a 300-page book. The system matches it against 300-page books. It returns 300-page books.
- Result: Computational chaos. No precision.

Separation of Concerns



Injecting Learnability: Linear Transformations

$$Q_i = \mathbf{E}_i \times \mathbf{W}_Q$$

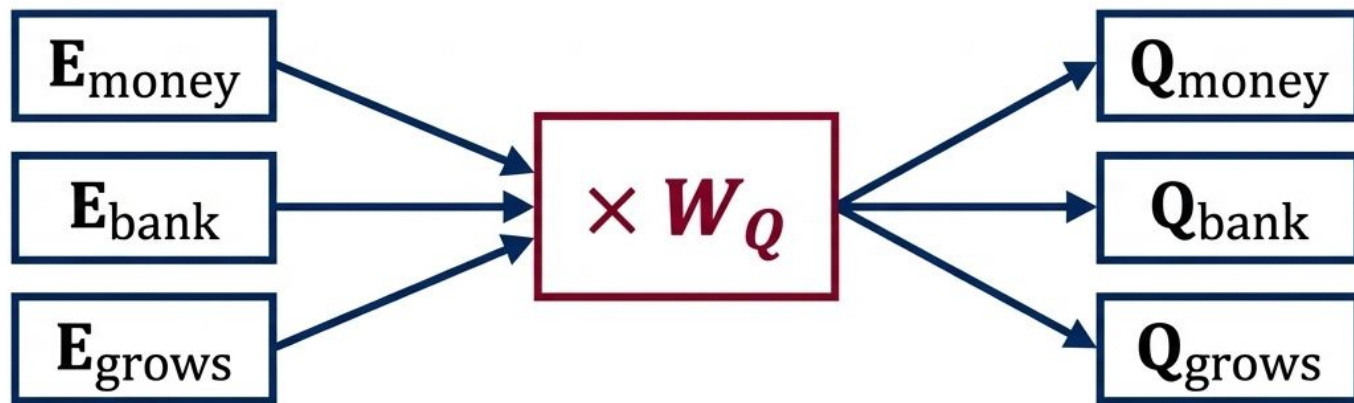
$$K_i = \mathbf{E}_i \times \mathbf{W}_K$$

$$V_i = \mathbf{E}_i \times \mathbf{W}_V$$

How Learning Occurs:

- We introduce three learnable weight matrices ($\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$).
- Initialized with random weights, these are optimized via backpropagation during training (e.g., on translation pairs).
- The network learns exactly *how* to transform static embeddings into the perfect interrogators, respondents, and payloads for a specific objective.

The QKV Generation Process



(Repeat this identical process with W_K to generate all Keys, and W_V to generate all Values).

Translational Invariance:

The mathematical rules for generating a Query or Key are **consistent** regardless of the word's word's **position** in the sequence. However, because the base E_i is **different**, the resulting **QKV vectors** are **entirely unique** to the word's semantic payload.

The Modern Self-Attention Mechanism

$$Y_i = \sum \text{Softmax}(\mathbf{Q}_i \cdot \mathbf{K}_j) \times \mathbf{V}_j$$

Query meets Key

Compute dot product of $\mathbf{Q}_i$ against all $\mathbf{K}_j$ to retrieve similarity scores.



Normalize

Apply Softmax to these raw scores to extract task-optimized probability weights.



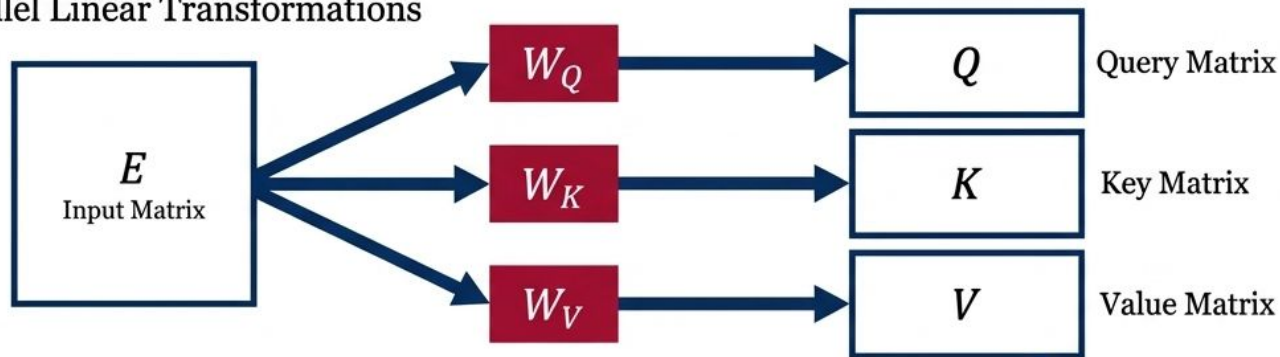
Extract Value

Multiply these learned weights by all $\mathbf{V}_j$ vectors and sum them up.

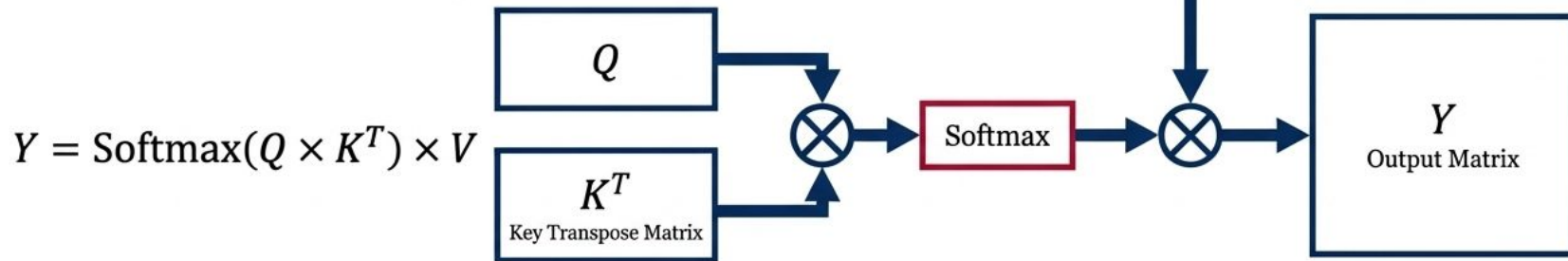
Because $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are derived from actively trained parameters, the final contextual vector $\mathbf{Y}_i$ is highly specific to the dataset's underlying logic.

Full Matrix Parallelization of QKV Attention

Phase 1: Parallel Linear Transformations



Phase 2: The Core Attention Equation (Parallelized)



The Result: In two lines of matrix algebra, we process thousands of words simultaneously, extracting profound, task-specific contextual relationships across an entire sequence. This formulation is the engine of modern Deep Learning.

Unscaled Self-Attention Formulation

From our first-principles derivation, Self-Attention computes contextual embeddings via the dot product of Queries (**Q**) and Keys (**K**), normalized by Softmax, and multiplied by Values (**V**).

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q} \mathbf{K}^T) \mathbf{V}$$



Computes raw alignment scores (scalar dot products) between every token.

Converts raw scores into a valid probability distribution (summing to 1).

Applies the learned attention weights to the value vectors to generate the final contextual embedding.